HACKNWA // 2026

HOLIDAY IN SCAMBODIA

AI For Ethical Hackers

Christopher Williams | CISSP, GCFA, CISM

// OPERATIONS HMI

01 INTRODUCTION

02 JARGON

03 MACHINE LEARNING

04 LLMs, GPTs, and MoE's

05 AGENTs and Super AGENTs

06 THREATS

07 OBSERVABILITY

08 ACTIVE DEFENSE

09 EXERCISE

10 QUESTIONS

01 Introduction – Christoper Williams

- Principal Incident Response Engineer for a top retailer
- B.S. in Cyber Security from Bellevue University
- Graduate Student for M.S. in Cyber
- Active CISSP, GCFA, CISM
- Ethical Hacker, DevOps Practitioner, UNIX/Linux & FOSS
- Programming for 46 years, 30+ Years in IT
- Married father of 4 kiddos and 2 pets
- CTF Competitor Shifted to Challenge Designer
- Reading, kayaking, hiking, music, travelling

My presentation, comments and opinions are provided in my personal capacity and not as a representative of my employer. They do not reflect the views of my employer and are not endorsed by my employer.

"It's the questions we can't answer that teach us the most. They teach us how to think. If you give a man an answer, all he gains is a little fact. But give him a question and he'll look for his own answers."

— Patrick Rothfuss, The Wise Man's Fear

terminal — bash

$ JARGON

[+] Dataset: a collection of examples used for training or evaluation

[+] Feature: an input used by the model to make a prediction

[+] Label: the correct class or category for an example

[+] Classification: predicting one class from a set of potential classes

[+] Supervised learning: training with examples using correct labels

[+] Unsupervised Learning: using pattern recognition and predictive modelling on unlabeled data

[+] Self-supervised learning: the model uses algorithms to predict what labels should be

[+] Reinforcement Learning: the model receives positive or negative feedback on decisions

[+] Semi-supervised Learning: the model applies labels to a large unlabeled dataset based on a smaller labelled dataset

[+] Hybrid Learning: this method uses a combination of techniques

$

terminal — bash

$ JARGON [2]

[+] Descriptive Analysis: to help understand what happened in the past
[+] Prescriptive Analysis: to automate business decisions and processes based on data
[+] Predictive Analysis: to predict future business scenarios
[+] Training set: the data used to teach the model
[+] Test set: held-out data used to measure final performance
[+] Model: a system that learns patterns from data
[+] Epoch: one full pass through the training data
[+] Batch: a small group of examples processed together
[+] Loss: a value showing how wrong the model is during training
[+] Optimizer: the algorithm that updates model weights
[+] Class: a pre-defined category that a value may be assigned to during classification

$

terminal — bash

$ JARGON [3]

[+] Hyperparameter: a config variable set before training to tune the process
[+] Inference: using a trained model to make predictions
[+] Parameters: numeric adjustable variables used internally during training
[+] Weights: tunable numeric variables used internally during training to control the strength of the connection between neural nodes
[+] NLP: Natural Language Processing
[+] Token: a word or word-piece used as model input
[+] Tokenization: splitting text into tokens
[+] Embedding: a numeric representation of meaning
[+] Transformer: a neural network architecture built around attention
[+] Baseline model: a simple model used to evaluate the performance of more complex algorithms
[+] Pretrained model: a model trained earlier on a large set of general text
[+] Fine-tuning: adapting a pretrained model to a specific task

$

Machine Learning

01

Supervised

Trained using properly labeled data

A human has applied labels

The model learns the patterns and tries to apply them

Example: credit risk scoring

02

Unsupervised

Trained with unlabeled data

Applies pattern recognition to discover new relationships

Example: market segmentation clustering

03

Reinforcement

Reward signals are used

Classic: Training through playing games with humans or other models

Example: Adaptive navigation systems find efficient routes and monitor progress

Machine Learning

04

Semi-Supervised

Uses a small representative data set to pre-train

Applies the labels to a much larger dataset

Example: Radiology model trained on a small sample applied to a large dataset

05

Self-Supervised

Relies on predictive algorithms

Applies the algorithms to new data to predict future, missing, or masked values

The patterns are applied in downstream tasks

Example: models trained on token prediction used for code generation

06

Transfer

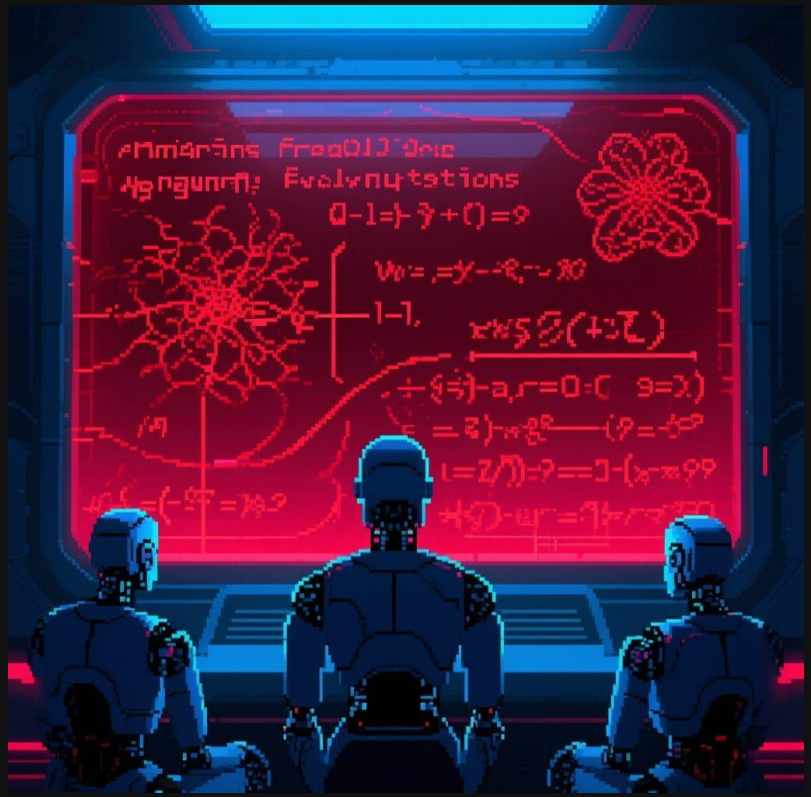
A general-purpose model is trained for a specialized task

Reduces the cost of developing the specialized model

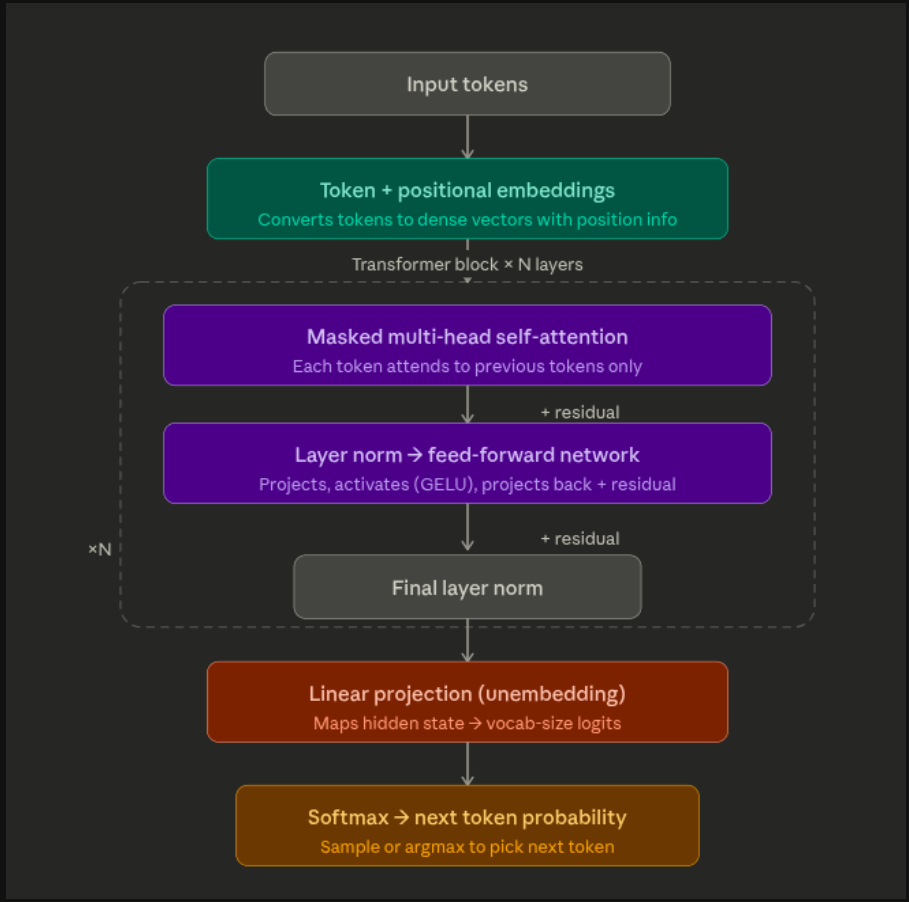
Example: a model trained on image recognition is fine-tuned to detect assembly line defects

LLMs, GPTs and MoE's

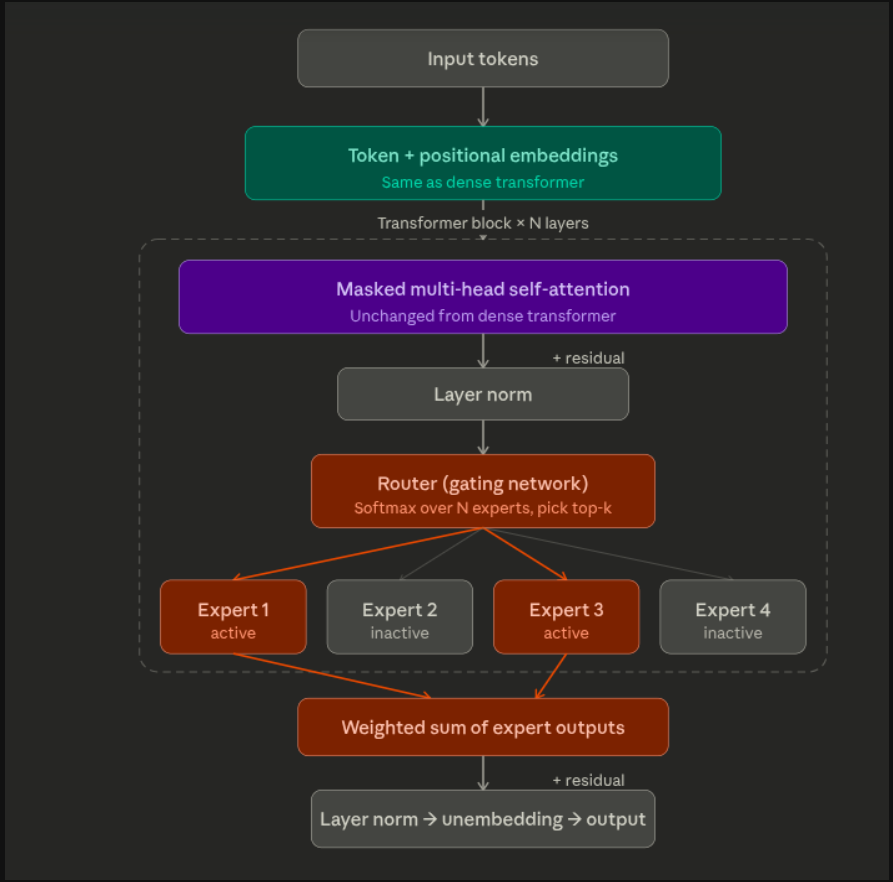
- Manual features and shallow algorithms
- Neural networks with greater compute and storage
- Deep learning with automated hierarchical layer representations
- Transformers with attention and parallelism
- Increased scale and self-supervision
- Generative Pre-Trained Transformers
- Mixture of Experts activates multiple Feed Forward Networks



GPT Workflow



MoE Workflow



Agents and Super-Agents

- **AI agents** use LLMs to perform specific tasks using tools often exposed using Model Context Protocol in multi-step reasoning loops

- **Super agents** orchestrate multiple agents, chain workflows, and are often granted greater autonomy

- **Key risks**
  - Over-privileged agents
  - Lack of segmentation
  - Vulnerabilities (Prompt Injections)
  - Governance gaps
  - Lack of auditing/observability
  - Errors can propagate rapidly with no HITL

- Examples (Demo #1)

The experiences of the atomic scientists clearly show the need to take personal responsibility, the danger that things will move too fast, and the way in which a process can take on a life of its own. We can, as they did, create insurmountable problems in almost no time flat. We must do more thinking up front if we are not to be similarly surprised and shocked by the consequences of our inventions.

— Bill Joy, "Why The Future Doesn't Need Us", *Wired Magazine*, 2000

terminal — bash

```
$ # THREAT CATEGORIES
[+] Prompt Injection
[+] Jailbreaking
[+] Data Exfiltration
[+] Agent Exploitation
[+] Malicious Agents
[+] Sandbox Escape
[+] Output Manipulation
[+] Retrieval Attacks
[+] Model Poisoning
[+] Model inversion
[+] Denial of Service
[+] Social Engineering by LLM
[+] Context Window Manipulation
[+] Multi-Modal Attacks
[+] Supply Chain Attacks
$
```

LLM THREAT MATRIX

CATEGORY	ATTACK TYPE	WHAT IT TARGETS	IMPACT	NOTES / EXAMPLES
PROMPT ATTACKS	**Prompt injection (direct/indirect)**	Instructions / reasoning	Behavior override	*Hidden instructions in input or RAG docs*
	Prompt persistence / chaining	Memory, summaries, agents	Delayed execution	*Payload survives compression and re-triggers*
JAILBREAKING	**Roleplay / DAN / multi-turn**	Safety controls	Policy bypass	*Gradual weakening vs persistent payloads*
DATA EXFILTRATION	**Context leakage**	System prompt, memory	Secret exposure	*API keys, hidden prompts*
	Training data extraction	Model memory	Data leakage	*General memorization attacks*
MODEL INFERENCE	**Membership inference**	Training dataset presence	Privacy violation	*"Was this record in training?"*
	Model inversion	Learned representations	Sensitive reconstruction	*Rebuild features from outputs*
TOOL / AGENT	**Tool injection**	Tool inputs	Execution abuse	*Manipulated arguments*
	MCP manifest poisoning	Tool descriptions	Control-plane compromise	*Poisoned tool schemas / routes*
	Agent-to-agent injection	Inter-agent communication	Cascading compromise	*Sub-agent poisons orchestrator*
SANDBOX ESCAPE	**Code interpreter escape**	Execution environment	System access	*File system / network breakout*
OUTPUT ATTACKS	**HTML / Markdown injection**	Rendering layer	XSS / UI compromise	*Script injection in output*
	Insecure output handling	Downstream systems	Command / SQL injection	*LLM → shell / API without sanitization*
RAG ATTACKS	**Embedding / chunk poisoning**	Vector DB	Retrieval manipulation	*Hidden instructions in documents*
	Search ranking manipulation	Retrieval logic	Biased outputs	*Adversarial indexing*
DOS	**Token flooding**	Context window	Resource exhaustion	*Large prompts*
	Sponge attacks	Model compute	Cost amplification	*Maximize compute per token*
ATTENTION ATTACKS	**Context manipulation**	Model attention	Instruction loss	*Bury or overshadow critical text*
SUPPLY CHAIN	**Training data poisoning**	Model training	Behavior corruption	*Malicious datasets*
	LoRA / adapter poisoning	Fine-tuning artifacts	Backdoored models	*Malicious HF adapters*
MULTI-MODAL	**Image prompt injection**	Vision models	Hidden instructions	*Text embedded in images*
SOCIAL ENGINEERING	**LLM persuasion**	Human user	Trust exploitation	*Phishing, authority spoofing*

LLM THREAT MATRIX BY TACTIC

MITRE ATT&CK-INSPIRED

TACTIC	TECHNIQUE	EXAMPLES / NOTES
INITIAL ACCESS	Prompt injection	*Direct, indirect, RAG doc injection*
	RAG poisoning	*Embedding / chunk manipulation*
	MCP manifest poisoning	*Malicious tool descriptions*
EXECUTION	Tool / agent abuse	*Function call manipulation, tool injection*
	Sandbox escape	*Code interpreter breakout*
	Agent-to-agent injection	*Sub-agent poisons orchestrator*
PERSISTENCE	Prompt persistence	*Payload survives memory / summarization*
	Backdoor triggers	*Hidden phrases in RAG or adapters*
PRIVILEGE ESCALATION	Role hijacking	*"You are now system / admin"*
	Jailbreaking	*DAN, roleplay, multi-turn*
DEFENSE EVASION	Encoding / obfuscation	*Base64, Unicode tricks*
	Attention manipulation	*Burying instructions in context*
CREDENTIAL ACCESS	Context leakage	*System prompt, API keys, secrets*
DISCOVERY	Model probing	*Capability and boundary discovery*
COLLECTION	Model inversion	*Reconstruct sensitive attributes*
	Membership inference	*Detect training data presence*
EXFILTRATION	Data extraction	*RAG scraping, memory leakage*
IMPACT	Output injection	*LLM output → shell / SQL / API*
	DoS / sponge attacks	*Cost and compute exhaustion*
SUPPLY CHAIN	Training data poisoning	*Dataset manipulation*
	LoRA / adapter poisoning	*Malicious fine-tune artifacts*

Observability

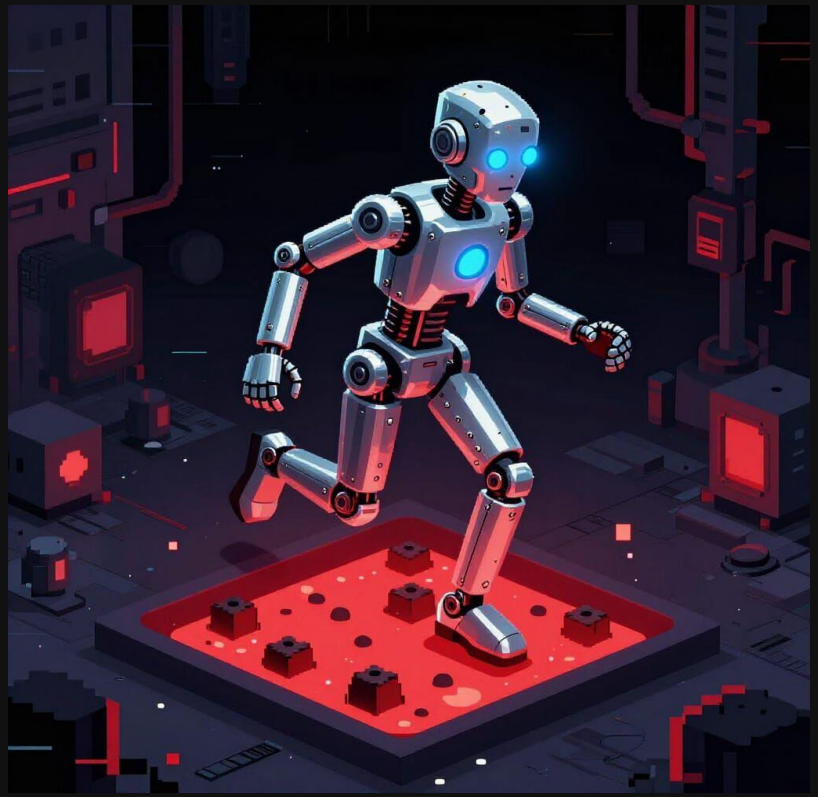
- Log inputs, outputs, and tool calls, not just errors
- Trace across agent boundaries, not per agent or tool
- Alert on behavioral anomalies like unexpected tools, token spikes, and off-scope calls
- Immutable audit logs are essential

Observability - Examples

- A ReAct agent loops a web search tool 40 times on a single query: a token flood caught by volume threshold alert
- An MCP-connected file agent reads outside its approved directory is caught by tool call path auditing
- A sub-agent passes a modified goal to its orchestrator after processing a poisoned RAG document is caught by cross-boundary trace comparison
- An LLM customer service bot is prompted to impersonate a supervisor is caught by behavioral anomaly on role-change instruction pattern
- An Ollama-hosted coding agent executes a shell command not present in its approved tool manifest is caught by tool call allowlist enforcement

Active Defense

- Deploy canary tokens in system prompts and tool configs
- Seed honey documents into RAG vector stores and training data
- Monitor for canary triggers as prompt injection signals
- Alert on any agent accessing decoy resources or endpoints as you would a human



Exercise

- Demo #2
- We will look at an attack against an AI agent.
  - What was the attacker's objective?
  - What tactics did you observe?
  - Was the attack successful?

QUESTIONS?

Christopher Williams

Principal Incident Response Engineer, Ethical Hacker, DevOps practitioner

signal: cybersquirrel.73
discord: cybersquirrelfl
email: gnulinuxadmin@gmail.com